DATA STRUCTURES – ASSIGNMENT

Register Number: 1911260093B

Name: Anand N

1. Define Data Structure and explain its importance in computing (5 Marks)

A data structure is a way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. Common examples include arrays, linked lists, stacks, queues, trees, graphs, and hash tables.

Data structures are important in computing for the following reasons:

1. Efficient data access: They help retrieve information quickly. For example, an array allows direct access using an index.

2. Better memory usage: Choosing a suitable structure helps store data effectively and reduce unnecessary memory use.

3. Faster operations: They improve the efficiency of searching, sorting, insertion, and deletion.

4. Managing complex data: Trees represent hierarchical data, while graphs represent connections such as computer networks.

5. Software development: Databases, operating systems, compilers, and many applications use data structures to manage information.

Thus, choosing the right data structure improves a program's speed, memory usage, and overall performance.

2. Develop a recursive solution for Binary Search (10 Marks)

Binary search is a searching technique used to find an element in a sorted array. It compares the target value with the middle element. If they are not equal, it searches only the half in which the target may be present. This process continues until the element is found or the search range becomes empty. Binary search requires the array to be sorted.

Algorithm:

```
RecursiveBinarySearch(A, low, high, key)
1. If low > high, return -1 (element not found).
2. Calculate mid = low + (high - low) / 2.
3. If A[mid] == key, return mid.
4. If key < A[mid], search the left half:
   RecursiveBinarySearch(A, low, mid - 1, key).
5. Otherwise, search the right half:
   RecursiveBinarySearch(A, mid + 1, high, key).
```

C Program:

```
#include <stdio.h>

int binarySearch(int a[], int low, int high, int key) {
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (a[mid] == key)
        return mid;
    else if (key < a[mid])
        return binarySearch(a, low, mid - 1, key);
    else
        return binarySearch(a, mid + 1, high, key);
}

int main(void) {
    int a[] = {10, 20, 30, 40, 50, 60, 70};
    int n = sizeof(a) / sizeof(a[0]);
    int key = 50;

    int result = binarySearch(a, 0, n - 1, key);

    if (result == -1)
        printf("Element not found\n");
    else
        printf("Element found at index %d\n", result);

    return 0;
}
```

Example: To search for 50 in the array {10, 20, 30, 40, 50, 60, 70}, the first middle element is 40, so the search continues in the right half. The next middle element is 60, so it searches the left part of that range. The next middle element is 50, so the element is found at index 4.

The time complexity of binary search is O(log n), because the search range is halved at every step. The auxiliary space complexity is O(log n) because of recursive function calls. If the element is not present, the function returns -1.

3. Explain ADT usage in Database Indexing (5 Marks)

An Abstract Data Type (ADT) defines a data type by describing the values it stores and the operations that can be performed on it, without specifying how those operations are implemented. A Dictionary or Map ADT, for example, supports operations such as insert, search, delete, and update using a key.

Usage of ADT in database indexing:

1. Fast searching: A database index helps locate records without scanning every row in a table.

2. Key-based access: The index maps a search key, such as a register number, to the location of the related record.

3. Insertion and deletion: Index operations update the key-to-record mapping when records are added or removed.

4. Implementation independence: The ADT describes the available operations, while the database system chooses how to implement them, for example using a B-tree or B+ tree.

5. Better query performance: Indexes can speed up searches and range queries, though they need extra storage and must be updated when data changes.

Example: In a student database, an index on register number helps locate a student's details quickly instead of checking every record. A B+ tree is a common data structure used to implement database indexes.

Therefore, ADTs provide a clear set of operations for database indexing, while structures such as B+ trees implement those operations efficiently.